MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO

GABRIEL DIAS VILELA
TASSANDRO CAVALCANTE LEITÃO

PLANEJAMENTO DE TRAJETÓRIA PARA ROBÔ TERRESTRE USANDO GRAFOS

RIO DE JANEIRO
2026

GABRIEL DIAS VILELA
TASSANDRO CAVALCANTE LEITÃO

PLANEJAMENTO DE TRAJETÓRIA PARA ROBÔ TERRESTRE USANDO GRAFOS

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador(es): Claudia Marcela Justel, D.Sc.
Paulo Fernando Ferreira Rosa, Ph.D.

Rio de Janeiro
2026

©2026
INSTITUTO MILITAR DE ENGENHARIA
Praça General Tibúrcio, 80 – Praia Vermelha
Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmar ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Dias Vilela, Gabriel; Cavalcante Leitão, Tassandro.

Planejamento de trajetória para robô terrestre usando grafos / Gabriel Dias Vilela e Tassandro Cavalcante Leitão. – Rio de Janeiro, 2026.

XX f.

Orientador(es): Claudia Marcela Justel e Paulo Fernando Ferreira Rosa.

Projeto de Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia da Computação, 2026.

1. Veículos terrestres não tripulados. 2. Segmentação semântica. 3. Algoritmos baseados em grafos. 4. Planejamento de trajetórias. 5. Robot Operating System. i. Justel, Claudia Marcela (orient.) ii. Ferreira Rosa, Paulo Fernando (orient.) iii. Título

GABRIEL DIAS VILELA
TASSANDRO CAVALCANTE LEITÃO

Planejamento de trajetória para robô terrestre usando grafos

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador(es): Claudia Marcela Justel e Paulo Fernando Ferreira Rosa.

Aprovada em ____ de _____ de 2026, pela seguinte banca examinadora:

Prof. Paulo Fernando Ferreira Rosa –
Professor do IME – Presidente

Profa. Claudia Marcela Justel – Professora
do IME

Membro 3 – Instituição

Membro 4 – Instituição

Membro 5 – Instituição

Rio de Janeiro
2026

Às nossas famílias.

Agradecimentos

Os agradecimentos principais são direcionados aos nossos orientadores por todo acompanhamento e ensinamentos. Agradecemos também ao Major Fábio Luiz Junior, cujos modelos de segmentação e trabalho de referência sobre planejamento em mapas de navegabilidade fundamentam este projeto, e aos autores do projeto anterior, João Vitor Camelo de Almeida e Thiago Calheiros de Souza Barbosa, cujo trabalho serviu de base e inspiração para o presente projeto.

"Não vos amoldeis às estruturas deste mundo,
mas transformai-vos pela renovação da mente,
a fim de distinguir qual é a vontade de Deus:
o que é bom, o que Lhe é agradável, o que é perfeito."
(Bíblia Sagrada, Romanos 12, 2)

Resumo

Este trabalho dá continuidade ao Projeto de Final de Curso de planejamento de trajetória para veículos terrestres não tripulados (UGV). Ao longo do presente ano, o projeto evoluiu de uma entrega inicial, que consolidou o planejamento do trabalho, a definição de uma arquitetura de *software* modular e orientada a objetos (organizada em cinco estágios encadeados: ingestão de dados, pré-processamento, construção de grafos, algoritmos e pós-processamento) e a implementação dos algoritmos de busca em grafos (BFS, DFS, Dijkstra e A*) sobre o cálculo de rotas na imagem, para a etapa atual, que consolida três contribuições: (i) a incorporação do algoritmo *Rapidly-exploring Random Tree* (RRT) ao catálogo de planejadores, permitindo comparar o planejamento baseado em amostragem com a busca em grafos; (ii) a integração, ao *pipeline* modular, de modelos de segmentação semântica pré-treinados baseados em aprendizado profundo (U-Net), treinados sobre o *Semantic Drone Dataset* (Institute of Computer Graphics and Vision, TU Graz, 2019) e disponibilizados em (JUNIOR, 2024b) (ResNet 50, ResNet 101, EfficientNet-B4 e EfficientNet-B5), para a identificação automática das áreas transitáveis; e (iii) a extensão do módulo de pós-processamento com uma ponte para o *Robot Operating System* (ROS 2) (QUIGLEY et al., 2009) e a simulação do movimento do robô no simulador Gazebo (KOENIG; HOWARD, 2004), na qual um UGV simulado (Clearpath Jackal) percorre efetivamente a trajetória planejada, fechando o ciclo entre planejamento e execução. As contribuições foram validadas sobre imagens RGB reais obtidas por UAV.

Palavras-chave: Veículos terrestres não tripulados; Segmentação semântica; Algoritmos baseados em grafos; Planejamento de trajetórias; RRT; ROS; Simulador Gazebo.

Abstract

This work continues the undergraduate final project on trajectory planning for unmanned ground vehicles (UGV). Over the present year, the project evolved from an initial delivery, which consolidated the project plan, the definition of a modular, object-oriented software architecture (organized into five chained stages: data ingestion, preprocessing, graph construction, algorithms, and post-processing), and the implementation of graph-search algorithms (BFS, DFS, Dijkstra, and A*) over image-based route computation, to the current stage, which consolidates three contributions: (i) the addition of the *Rapidly-exploring Random Tree* (RRT) algorithm to the planner catalog, enabling a comparison between sampling-based planning and graph search; (ii) the integration, into the modular *pipeline*, of pre-trained deep-learning semantic segmentation models (U-Net), trained on the *Semantic Drone Dataset* (Institute of Computer Graphics and Vision, TU Graz, 2019) and released in (JUNIOR, 2024b) (ResNet 50, ResNet 101, EfficientNet-B4, and EfficientNet-B5), for the automatic identification of traversable areas; and (iii) the extension of the post-processing module with a bridge to the *Robot Operating System* (ROS 2) (QUIGLEY et al., 2009) and the simulation of the robot's motion in the Gazebo simulator (KOENIG; HOWARD, 2004), where a simulated UGV (Clearpath Jackal) actually drives along the planned trajectory, closing the loop between planning and execution. The contributions were validated on real RGB imagery acquired by a UAV.

Keywords: Unmanned ground vehicles; Semantic segmentation; Graph-based algorithms; Trajectory planning; RRT; ROS; Gazebo simulator.

Lista de ilustrações

Figura 1 – Pipeline do sistema proposto. Em relação à primeira versão, o RRT foi acrescentado ao módulo `algorithm` e uma ponte `ros_bridge` (ROS 2) foi inserida entre o pós-processamento e a simulação no Gazebo. 25
Figura 2 – Comparativo da estrutura entre o PFC 2025 (ALMEIDA; BARBOSA, 2025) e esta versão. Em destaque, os três aprimoramentos: segmentação semântica, inclusão do RRT e simulação em ciclo fechado. 26
Figura 3 – Segmentação com o modelo EfficientNet-B4: em verde, a região classificada como transitável, sobreposta à imagem aérea de entrada. 32
Figura 4 – UGV Clearpath Jackal simulado em um mundo do Gazebo. À direita, o painel de teleoperação publicando comandos de velocidade (`geometry_msgs/Twist`) no tópico `cmd_vel` consumido pela integração. 34
Figura 5 – Cronograma de atividades 36

Lista de tabelas

Tabela 1 – Modelos de segmentação semântica disponibilizados em (JUNIOR, 2024b). 21
Tabela 2 – Comparação dos modelos de segmentação. mIoU e acurácia conforme (JUNIOR, 2024b); área transitável e tempo de inferência medidos neste trabalho (1024×768, *backend* MPS). 31
Tabela 3 – Comparação dos algoritmos sobre o mesmo mapa (1000×750; mesmos início e fim). †Para o RRT, o custo é o comprimento geométrico, não comparável ao custo ponderado dos demais. 32

Lista de abreviaturas e siglas

BFS	Breadth-First Search
CNN	Convolutional Neural Network
CTEx	Centro Tecnológico do Exército
DDS	Data Distribution Service
DFS	Depth-First Search
EB	Exército Brasileiro
EKF	Extended Kalman Filter
GSD	Ground Sampling Distance
IA	Inteligência Artificial
mIoU	Mean Intersection over Union
OOP	Object-Oriented Programming
QoS	Quality of Service
ROS	Robot Operating System
RRT	Rapidly-exploring Random Tree
SDD	Semantic Drone Dataset
SVTRP	Sistemas de Veículos Terrestres Remotamente Pilotados
UAV	Unmanned Aerial Vehicle
UGV	Unmanned Ground Vehicle

Sumário

1 INTRODUÇÃO 14
1.1 Da Primeira à Segunda Versão deste Projeto 15
1.2 Motivação e Justificativa 16
1.3 Objetivos 16
1.4 Metodologia 17
1.5 Estrutura 17

2 CONCEITOS PRELIMINARES 18
2.1 Veículos Terrestres Autônomos 18
2.2 Planejamento de Caminhos 18
2.3 Grafos 18
2.4 Algoritmos de Caminhos Mínimos em Grafos 19
2.4.1 Breadth-First Search (BFS) 19
2.4.2 Depth-First Search (DFS) 19
2.4.3 Algoritmo de Dijkstra 20
2.4.4 Algoritmo A* 20
2.5 Árvore de Exploração Rápida Aleatória (RRT) 20
2.6 Segmentação Semântica e Identificação de Áreas Transitáveis . . . 21
2.6.1 Modelos Pré-treinados Adotados 21
2.6.2 Métrica mIoU 22
2.6.3 Geração do Mapa de Navegabilidade 22
2.7 ROS e Simulação Robótica 22
2.7.1 Robot Operating System (ROS 2) 22
2.7.2 Simulador Gazebo 23

3 METODOLOGIA 24
3.1 Arquitetura do Sistema 24
3.1.1 Diferenças em Relação à Primeira Versão e ao PFC 2025 25
3.2 Construção dos Grafos 26
3.3 Aprimoramento 1: Inclusão do Algoritmo RRT 27
3.4 Aprimoramento 2: Implementação da Segmentação Semântica . . . 28
3.5 Aprimoramento 3: Integração com ROS 2 e Simulação no Gazebo . 29
3.5.1 Publicação: a ponte `ros_bridge` 29
3.5.2 Consumo: o pacote `ugv_bridge` e o UGV simulado 29
3.6 Pós-processamento e Formato de Saída 30
3.7 Coleta dos Dados 30

4 RESULTADOS E DISCUSSÃO 31
4.1 Comparação dos Modelos de Segmentação 31
4.2 Desempenho dos Algoritmos de Planejamento 32
4.3 Validação no Simulador Gazebo 33

5 CONCLUSÃO 35
5.1 Perspectivas Futuras 35

6 CRONOGRAMA 36

REFERÊNCIAS 37

1 INTRODUÇÃO

Os avanços tecnológicos envolvendo visão computacional, eletrônica e inteligência artificial, entre outras áreas correlatas, impulsionaram a tendência de desenvolvimento e uso de ferramentas que automatizam as disputas em ambientes de combate. Nesse sentido, com a possibilidade de aumentar a eficácia operacional e reduzir a participação e desgaste humano (física, cognitiva e psicologicamente) no campo de batalha, a integração de robôs autônomos nesses cenários de confronto é iminente e torna-se motivo de vantagem tática e estratégica.

Em relação à autonomia desses sistemas, o planejamento de trajetória de robôs terrestres é uma das tarefas essenciais a ser realizada e ela se encontra dentro de um escopo multidisciplinar que inclui conhecimentos em diversas áreas de grande complexidade, como a robótica e a teoria dos grafos. Essas disciplinas convergem na modelagem do cenário de combate como um grafo, no qual os nós representam as posições transitáveis pelos robôs no mapa e as arestas definem os caminhos. Dessa maneira, algoritmos de busca e otimização são ferramentas que guiam o planejamento das possíveis rotas, auxiliando na coordenação entre pontos e adaptação aos obstáculos.

A pesquisa "Cooperação Entre Plataformas Terrestres e Aéreas para Aquisição, Rastreamento e Perseguição de Alvos, Utilizando Visão Computacional e Inteligência Artificial" (JUNIOR, 2024a) que se encontra em fase de desenvolvimento no laboratório LIARC, iniciada em 2023 pelo Major Fábio Luiz Junior, aborda o emprego de plataformas robóticas aéreas (UAV) e terrestres (UGV) em ambientes de batalha e como essas duas estruturas podem trabalhar em conjunto para otimizar e ampliar o poder de ataque e estratégico militar.

Contribuições do Projeto Anterior

O presente projeto é uma continuação direta do trabalho "Planejamento de Trajetória para Robô Terrestre, Utilizando Otimização em Grafos", desenvolvido em 2025 por Camelo de Almeida e Calheiros de Souza Barbosa (ALMEIDA; BARBOSA, 2025). Naquele trabalho, os autores propuseram uma abordagem baseada em grafos para o planejamento de caminhos sobre imagens aéreas segmentadas semanticamente, implementando os algoritmos de Dijkstra e A* e comparando-os com o algoritmo estocástico RRT.

As principais conclusões do projeto de 2025 foram:

- Os algoritmos baseados em grafos (Dijkstra e A*) apresentaram desempenho mais consistente, confiável e previsível em relação ao RRT, destacando-se pelo determinismo e pela garantia de otimalidade das soluções;

- O algoritmo A* mostrou-se ligeiramente mais rápido que o Dijkstra, explorando um menor número de nós, sendo considerado mais adequado ao problema proposto;
- A inserção de pesos nas arestas do grafo, com base na proximidade a obstáculos, produziu caminhos mais realistas que se distanciam de regiões de risco, sem impacto significativo no tempo de execução;
- O RRT apresentou dificuldades em imagens com áreas transitáveis estreitas, excedendo o tempo limite em dois dos cinco cenários testados;
- A abordagem proposta, ao integrar segmentação semântica de imagens e planejamento baseado em grafos, constitui uma alternativa promissora para aplicações em veículos autônomos.

1.1 Da Primeira à Segunda Versão deste Projeto

Este Projeto de Final de Curso foi conduzido, até então, em duas entregas. A **primeira versão** teve caráter de planejamento e de estruturação: definiu o escopo do trabalho, projetou a arquitetura de *software* modular e orientada a objetos (cinco módulos encadeados, com interfaces abstratas e padrões de fábrica) e implementou os algoritmos-base de busca em grafos (BFS, DFS, Dijkstra e A*) sobre um sistema de cálculo de rotas diretamente na imagem, tomando como entrada um mapa de navegabilidade com cores fixas. Naquele estágio, a segmentação semântica e a validação em simulação foram apresentadas como *propostas* de arquitetura, ainda não implementadas.

Esta **segunda versão** concentra-se em transformar aquelas propostas em componentes efetivamente implementados e integrados ao *pipeline*, além de ampliar o catálogo de planejadores. Os três aprimoramentos centrais são:

1. **Inclusão do algoritmo RRT.** O planejador baseado em amostragem *Rapidly-exploring Random Tree* (LAVALLE, 1998) foi adicionado ao catálogo de algoritmos, ao lado de BFS, DFS, Dijkstra e A*, sob a mesma interface. Isso permite contrastar diretamente, sobre os mesmos mapas, o comportamento de um planejador estocástico com o dos algoritmos determinísticos de busca em grafos, retomando de forma integrada a comparação iniciada em (ALMEIDA; BARBOSA, 2025).
2. **Implementação da segmentação semântica.** A identificação das áreas transitáveis, antes dependente de mapas previamente segmentados, passou a ser realizada dentro do próprio *pipeline* por meio dos modelos U-Net pré-treinados disponibilizados em (JUNIOR, 2024b), treinados sobre o *Semantic Drone Dataset* (Institute of Computer Graphics and Vision, TU Graz, 2019). O módulo de pré-processamento executa a rede sobre a imagem RGB do UAV e produz um mapa de navegabilidade pronto para a conversão em grafo.

3. **Integração com ROS 2 e simulação do movimento no Gazebo.** O módulo de pós-processamento foi estendido com uma ponte para o ROS 2 (QUIGLEY et al., 2009), que publica a imagem anotada e a trajetória calculada; do lado do robô, um conjunto de nós ROS 2 faz um UGV simulado (Clearpath Jackal) percorrer efetivamente essa trajetória no Gazebo (KOENIG; HOWARD, 2004). Deixa-se, assim, de apenas "produzir *waypoints* compatíveis" para, de fato, *simular o movimento do robô* ao longo do caminho planejado.

O restante deste documento descreve esses aprimoramentos, dando ênfase às diferenças em relação à primeira versão e ao projeto de 2025 (ALMEIDA; BARBOSA, 2025).

1.2 Motivação e Justificativa

O presente projeto busca dar continuidade e aprofundar o estudo de planejamento de rotas abordado em (ALMEIDA; BARBOSA, 2025) e (JUNIOR, 2024a) e que, por sua vez, poderá ser aproveitado e aplicado no projeto estratégico "Sistemas de Veículos Terrestres Remotamente Pilotados" (SVTRP), conduzido pelo Centro Tecnológico do Exército (CTEx), visando a uma potencialização de seu funcionamento.

O projeto anterior demonstrou a viabilidade da abordagem baseada em grafos, porém dependia de imagens previamente segmentadas e não contemplava a execução dos caminhos em um robô. A necessidade de integrar o processo de segmentação diretamente no *pipeline*, utilizando modelos especializados em imagens de drones, e de validar os caminhos em um veículo simulado constitui a principal motivação desta segunda versão. A adoção dos modelos pré-treinados disponibilizados em (JUNIOR, 2024b) permite reutilizar diretamente o conhecimento já validado em pesquisa anterior do laboratório (JUNIOR, 2024a), evitando o re-treinamento e mantendo coerência com a linha de trabalho do CTEx.

Adicionalmente, a inclusão do RRT ao catálogo dá base experimental para discutir, sobre os mesmos cenários, os compromissos entre planejadores determinísticos e estocásticos; e a integração com o Gazebo, motivada pelo objetivo de validar os caminhos antes de qualquer implantação em campo, fecha o ciclo entre o planejamento e a execução do movimento em um UGV.

1.3 Objetivos

O objetivo principal desta segunda versão consiste em consolidar, sobre a arquitetura modular definida anteriormente, os três aprimoramentos descritos na Seção 1.1, obtendo um sistema capaz de, a partir de uma imagem aérea real, identificar áreas transitáveis, planejar uma trajetória por diferentes algoritmos e simular a sua execução em um UGV.

Para atingir o objetivo principal, foram definidos os seguintes objetivos específicos:

- Incorporar o algoritmo RRT ao módulo de algoritmos, sob a mesma interface dos demais planejadores, e compará-lo com BFS, DFS, Dijkstra e A* sobre os mesmos mapas;
- Implementar o módulo de segmentação semântica com os modelos pré-treinados de (JUNIOR, 2024b), gerando mapas de navegabilidade no formato adotado em (JUNIOR, 2025) a partir de imagens RGB reais de drone;
- Estender o pós-processamento com uma ponte ROS 2 e integrar um UGV simulado (Clearpath Jackal) no Gazebo; e
- Executar testes comparativos, de modelos de segmentação e de algoritmos de planejamento, sobre imagens RGB reais obtidas por drone.

1.4 Metodologia

Visando alcançar os objetivos propostos, a metodologia adotada consiste nas seguintes etapas. Inicialmente, foi conduzida uma revisão do projeto anterior (ALMEIDA; BARBOSA, 2025) e da literatura relacionada a segmentação semântica de imagens aéreas, planejamento de trajetórias e simulação robótica. Em seguida, sobre a arquitetura modular já definida na primeira versão, foram implementados os três aprimoramentos: o algoritmo RRT foi adicionado ao módulo `algorithm`; o módulo `preprocessing` passou a executar os modelos de segmentação de (JUNIOR, 2024b); e o módulo `post_processing` foi acoplado a uma ponte ROS 2 que alimenta a simulação no Gazebo. Por fim, foram executados testes sobre imagens RGB reais obtidas por drone, comparando os diferentes modelos de segmentação e algoritmos de planejamento, e verificando a execução da trajetória no UGV simulado.

1.5 Estrutura

Este documento está organizado da seguinte forma. No Capítulo 2, são apresentados os conceitos preliminares que fundamentam o projeto, incluindo grafos, algoritmos de caminhos mínimos, o planejamento por amostragem (RRT), a segmentação semântica e a simulação robótica com ROS e Gazebo. No Capítulo 3, descreve-se a arquitetura do sistema e o detalhamento dos três aprimoramentos desta versão. O Capítulo 4 é dedicado à apresentação e discussão dos resultados experimentais. No Capítulo 5, são apresentadas as conclusões e as perspectivas futuras. Por fim, o Capítulo 6 apresenta o cronograma das atividades.

2 CONCEITOS PRELIMINARES

Neste capítulo, apresentamos os conceitos fundamentais necessários para a compreensão do projeto. Além dos tópicos já abordados no projeto anterior (ALMEIDA; BARBOSA, 2025) (veículos terrestres autônomos, planejamento de caminhos, grafos e algoritmos clássicos), introduzimos os conceitos relacionados aos três aprimoramentos desta versão: o planejamento por amostragem (RRT), a segmentação semântica e a simulação robótica com ROS e Gazebo.

2.1 Veículos Terrestres Autônomos

Segundo (FINN; SCHEDING, 2010), um veículo terrestre autônomo (UGV) é um sistema robótico complexo e multidisciplinar, com participação ostensiva das engenharias mecânica, da computação e eletrônica. Este sistema tem como objetivo navegar pelo terreno de forma autônoma, sem a necessidade de intervenção por um operador externo, e executar determinada tarefa. Nos últimos anos, evidencia-se grande crescimento nas aplicações de UGVs, desde aplicações civis até aplicações militares.

2.2 Planejamento de Caminhos

O planejamento de caminhos é considerado um desafio crítico em Inteligência Artificial e essencial para o movimento autônomo de todos os sistemas inteligentes, como veículos terrestres (MADRIDANO et al., 2021). O planejamento pode ser categorizado como global (estático), quando toda a informação sobre o ambiente é conhecida, ou local (dinâmico), quando o ambiente é parcialmente desconhecido (CHEN; ZHAO; YIN, 2020). Neste trabalho, abordamos o planejamento de caminhos globais.

2.3 Grafos

Um grafo $G = (V, E)$ consiste em um conjunto finito de vértices $V = \{v_1, v_2, \dots v_n\}$ e um conjunto finito de arestas $E = \{e_1, e_2, \dots e_m\}$, com $m, n \in N$, tal que cada aresta é identificada com um par de vértices (v_i, v_j) (CHEN, 1997). Podemos classificar os grafos como ponderados, caso suas arestas possuam pesos associados, ou não ponderados, caso contrário. A representação de grafo enriquecida adotada neste projeto, com atributos associados a nós e arestas, é detalhada na Seção 3.2, junto à construção do grafo.

2.4 Algoritmos de Caminhos Mínimos em Grafos

A definição de caminho mínimo entre dois vértices apoia-se em duas noções básicas da teoria dos grafos (DEO, 2016; CHEN, 1997). Um *caminho* de um vértice u para um vértice v em um grafo $G = (V, E)$ é uma sequência alternada de vértices e arestas

$$P = (v_0, e_1, v_1, e_2, v_2, \ldots, e_k, v_k),$$

com $v_0 = u$, $v_k = v$, $e_i = (v_{i-1}, v_i) \in E$ para todo i e sem repetição de vértices. O *comprimento* do caminho P em um grafo ponderado é a soma dos pesos das suas arestas:

$$\ell(P) = \sum_{i=1}^{k} w(e_i),$$

onde $w : E \to \mathbb{R}_{\geq 0}$ é a função de pesos do grafo. No caso não ponderado, define-se $w(e_i) = 1$ para toda aresta, de modo que o comprimento coincide com o número de arestas do caminho.

O *problema do caminho mínimo* entre u e v consiste, então, em determinar um caminho P^* tal que

$$\ell(P^*) = \min\{\, \ell(P) : P \text{ é caminho de } u \text{ a } v \,\}.$$

No contexto deste projeto, o grafo é construído a partir do mapa de navegabilidade gerado pela segmentação semântica, e o caminho mínimo corresponde à trajetória de menor custo entre o ponto inicial e o ponto final escolhidos pelo operador.

A seguir, são apresentados os quatro algoritmos clássicos de busca em grafos implementados neste projeto.

2.4.1 Breadth-First Search (BFS)

A BFS é um algoritmo que permite percorrer vértices e arestas em grafos (CORMEN et al., 2009; MOORE, 1959). Iniciando o percurso em um nó denominado fonte, a busca se expande visitando os nós em uma mesma profundidade antes de avançar. Sua particularidade, buscar em largura, permite determinar caminhos mínimos em grafos não ponderados de maneira eficiente, com complexidade $O(|V| + |E|)$.

2.4.2 Depth-First Search (DFS)

Diferentemente da BFS, a DFS (CORMEN et al., 2009) explora o grafo percorrendo cada ramo o mais profundamente possível antes de retroceder. Embora não garanta encontrar o caminho mínimo, é útil em diversas aplicações em que a existência de qualquer caminho é mais relevante que sua otimalidade, por exemplo, determinar a conexidade do grafo. Por essas razões, a DFS é incluída neste trabalho como base de comparação para os demais algoritmos.

2.4.3 Algoritmo de Dijkstra

O algoritmo de Dijkstra (DIJKSTRA, 1959) encontra o caminho mínimo em grafos com pesos não negativos, construindo os caminhos mínimos de forma iterativa, a partir da fonte e na ordem crescente de distância. A cada iteração, seleciona-se o vértice ainda não processado com menor distância acumulada para ser expandido, atualizando-se as estimativas de seus vizinhos.

2.4.4 Algoritmo A*

O algoritmo A* (HART; NILSSON; RAPHAEL, 1968) é um algoritmo de busca informada que utiliza uma função de avaliação $f(n) = g(n) + h(n)$, onde $g(n)$ é o custo do caminho da fonte até n e $h(n)$ é uma estimativa heurística do custo restante até o destino. Por se basear em uma estimativa heurística do custo até o objetivo, o A* é, em geral, um algoritmo aproximado: a qualidade da solução depende da heurística utilizada e, no caso geral, não há garantias de mínimo global. Para uma heurística admissível h (isto é, que nunca superestime o custo real até o objetivo), o A* recupera essa propriedade e garante otimalidade, explorando o menor número necessário de nós. Neste projeto, utiliza-se a distância euclidiana como heurística, que é admissível para grafos em grade com pesos não negativos.

2.5 Árvore de Exploração Rápida Aleatória (RRT)

Diferentemente dos algoritmos de busca em grafos apresentados na Seção 2.4, que operam sobre uma estrutura discreta e (nos casos de Dijkstra e A*) garantem otimalidade, o *Rapidly-exploring Random Tree* (RRT) (LAVALLE, 1998; XU, 2024) é um planejador **baseado em amostragem**, que constrói incrementalmente uma árvore explorando aleatoriamente o espaço livre. A partir de um nó raiz (posição inicial), a cada iteração o algoritmo: (i) amostra um ponto aleatório no espaço; (ii) identifica o nó da árvore mais próximo desse ponto; (iii) avança uma distância fixa (*step size*) na direção do ponto amostrado, gerando um novo nó candidato; e (iv) adiciona o novo nó à árvore caso o segmento que o conecta ao nó mais próximo esteja livre de colisões. O processo termina quando um novo nó possui linha de visada desobstruída até o objetivo.

O RRT é *probabilisticamente completo* (a probabilidade de encontrar uma solução, se ela existir, tende a um à medida que o número de amostras cresce), mas **não é ótimo**: o caminho encontrado depende da sequência de amostras e tende a ser irregular, não correspondendo, em geral, ao caminho de menor comprimento. Por essa natureza estocástica, ele *não* se enquadra na formulação de caminho mínimo da Seção 2.4, sendo aqui utilizado como uma abordagem alternativa e como base de comparação, em linha com a avaliação já realizada em (ALMEIDA; BARBOSA, 2025). Sua principal vantagem é o baixo custo

por iteração, o que costuma resultar em tempos de planejamento reduzidos; sua principal limitação, já observada no projeto anterior, é a dificuldade em regiões transitáveis estreitas.

2.6 Segmentação Semântica e Identificação de Áreas Transitáveis

A segmentação semântica consiste em atribuir uma classe (rótulo) a cada pixel de uma imagem. Diferentemente da classificação de imagens, que atribui um rótulo único à imagem inteira, a segmentação semântica produz um mapa denso de classes com a mesma resolução da imagem de entrada. No contexto deste projeto, ela é utilizada para identificar automaticamente as regiões transitáveis do terreno a partir de imagens aéreas reais, em substituição às abordagens baseadas em segmentação por cor.

Diferentemente do projeto anterior (ALMEIDA; BARBOSA, 2025), no qual eram exploradas estratégias de segmentação por limiarização no espaço HSV, optou-se neste trabalho por uma abordagem baseada nos modelos pré-treinados disponibilizados em (JUNIOR, 2024b). Esses modelos foram desenvolvidos no âmbito da pesquisa de Junior (JUNIOR, 2024a) e são treinados especificamente sobre o *Semantic Drone Dataset* (Institute of Computer Graphics and Vision, TU Graz, 2019), conjunto composto por imagens aéreas anotadas pixel a pixel.

2.6.1 Modelos Pré-treinados Adotados

O repositório (JUNIOR, 2024b) disponibiliza modelos de segmentação semântica avaliados no conjunto de teste do *Semantic Drone Dataset* em termos de *Mean Intersection over Union* (mIoU) e acurácia por pixel, conforme a Tabela 1. Os modelos seguem a arquitetura U-Net (RONNEBERGER; FISCHER; BROX, 2015), com diferentes *backbones* convolucionais (ResNet 50 e ResNet 101) e baseados em redes com escalonamento composto (EfficientNet-B4 e B5) (JUNIOR, 2024a). A escolha do modelo em tempo de execução é, portanto, um hiperparâmetro do *pipeline*, permitindo equilibrar acurácia e tempo de inferência.

Tabela 1 – Modelos de segmentação semântica disponibilizados em (JUNIOR, 2024b).

Modelo	**mIoU**	**Acurácia**
ResNet 50	0,3933	0,8244
ResNet 101	0,4032	0,8720
EfficientNet-B4	0,5012	0,9120
EfficientNet-B5	0,4819	0,9025

2.6.2 Métrica mIoU

A *Intersection over Union* (IoU) de uma classe é a razão entre a interseção e a união das regiões prevista e anotada, equivalente a IoU = TP/(TP+FP+FN). A mIoU é a média das IoUs sobre todas as classes. Ela é preferível à acurácia por pixel porque esta última é inflada por classes dominantes (por exemplo, uma imagem majoritariamente composta por grama). Sobre o *Semantic Drone Dataset*, que possui múltiplas classes, valores de mIoU em torno de 0,5 são considerados um resultado razoável.

2.6.3 Geração do Mapa de Navegabilidade

A saída da segmentação é processada para gerar um *mapa de navegabilidade* no formato consolidado em (JUNIOR, 2025): uma imagem na qual as regiões transitáveis recebem cores específicas, enquanto as não transitáveis (vegetação, edificações, obstáculos) recebem cores distintas. Esse mapa serve como entrada direta para o módulo de construção de grafos. Em particular, adota-se o mesmo critério de (JUNIOR, 2025): as áreas transitáveis são representadas pelas cores BGR $(84, 1, 68)$ e $(0, 0, 255)$, e a verificação de colisão entre dois pontos é feita inspecionando as cores ao longo da reta que os conecta.

2.7 ROS e Simulação Robótica

2.7.1 Robot Operating System (ROS 2)

O *Robot Operating System* (ROS) (QUIGLEY et al., 2009) é um *framework* de código aberto amplamente utilizado no desenvolvimento de *software* robótico. Ele organiza o sistema em *nós* (processos) que se comunicam por *tópicos*, publicando e assinando mensagens tipadas. O ROS distribui um amplo conjunto de mensagens padronizadas, organizadas em pacotes por domínio, como `std_msgs` (tipos básicos), `sensor_msgs` (leituras de sensores, como imagens e nuvens de pontos), `geometry_msgs` (poses, vetores e velocidades) e `nav_msgs` (dados de navegação, como mapas e trajetórias), entre muitos outros; essa padronização favorece a interoperabilidade entre componentes de diferentes fornecedores. Neste projeto, são utilizadas duas dessas mensagens: `sensor_msgs/Image`, para a imagem aérea anotada com a trajetória, e `nav_msgs/Path`, para a sequência de poses (*waypoints*) que descreve o caminho. O ROS 2 realiza a comunicação sobre o *Data Distribution Service* (DDS), cujas políticas de *Quality of Service* (QoS) permitem, por exemplo, reter a última mensagem publicada para assinantes que se conectem tardiamente (durabilidade *transient local*).

2.7.2 Simulador Gazebo

O Gazebo (KOENIG; HOWARD, 2004) é um simulador robótico de código aberto amplamente utilizado pela comunidade de robótica móvel, com integração nativa ao ROS. Ele permite modelar com elevado grau de fidelidade a dinâmica de veículos, sensores embarcados e o ambiente de operação (incluindo terrenos, obstáculos e fontes de luz) antes de qualquer implantação em campo. Em sistemas de UGVs, o Gazebo é tipicamente empregado para validar pilhas de navegação completas, expondo as mesmas interfaces (tópicos ROS, mensagens `nav_msgs/Path`, comandos de velocidade `geometry_msgs/Twist`, odometria `nav_msgs/Odometry`) que seriam utilizadas em um robô real. No *pipeline* deste trabalho, o Gazebo ocupa o último estágio: a trajetória calculada é entregue, via ROS 2, a um UGV simulado (modelo Clearpath Jackal) que a percorre efetivamente, oferecendo uma camada adicional de validação antes da execução em campo.

3 METODOLOGIA

Neste capítulo, apresentamos a arquitetura do sistema e detalhamos os três aprimoramentos que caracterizam esta segunda versão: a inclusão do RRT, a implementação da segmentação semântica e a integração com ROS 2 e Gazebo.

3.1 Arquitetura do Sistema

O sistema foi projetado seguindo princípios de orientação a objetos, com cinco módulos independentes e encadeados, cada um definido por interfaces abstratas e instanciado por fábricas (*Factory Pattern*):

1. **`data_ingestion`**: responsável pela ingestão e estruturação dos dados brutos. Define uma interface genérica (`IngestedData`) com especialização para imagens (`ImageIngestedData`) capturadas pelo UAV;

2. **`preprocessing`**: aplica a segmentação semântica sobre a imagem ingerida, identificando os pixels transitáveis e produzindo um `SegmentedImageData` enriquecido com a máscara de navegabilidade;

3. **`graph_parsing`**: converte o mapa de navegabilidade em uma estrutura de grafo genérica (`Graph`), com *node embeddings*, *edge index* e *edge embeddings*;

4. **`algorithm`**: executa algoritmos de planejamento sobre o grafo (BFS, DFS, Dijkstra, A* e, nesta versão, RRT), retornando um `PathResult` padronizado com a sequência de nós, custo total e metadados;

5. **`post_processing`**: converte o resultado em *waypoints* para o robô (`RobotPath`) e em visualizações anotadas, além de publicar imagem e trajetória para o ROS 2 (Seção 3.5).

Cada módulo possui um arquivo `interfaces.py` com classes abstratas, `entities.py` com *dataclasses*, `index.py` com a fábrica de registro, e um `__init__.py` com as exportações públicas. A Figura 1 ilustra o *pipeline* completo, agora incluindo o RRT entre os algoritmos e a ponte ROS 2 que alimenta o Gazebo.

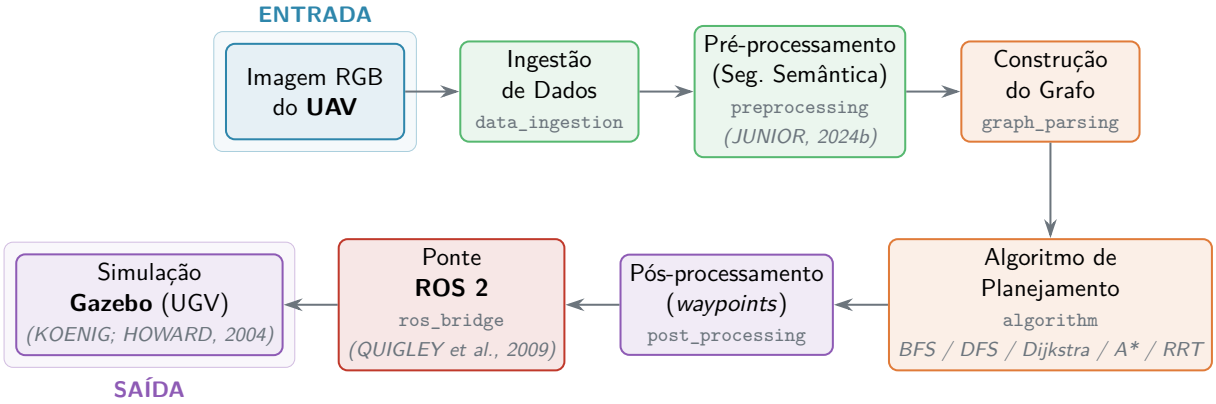


Figura 1 – Pipeline do sistema proposto. Em relação à primeira versão, o RRT foi acrescentado ao módulo `algorithm` e uma ponte `ros_bridge` (ROS 2) foi inserida entre o pós-processamento e a simulação no Gazebo.

3.1.1 Diferenças em Relação à Primeira Versão e ao PFC 2025

Em relação ao trabalho de Camelo de Almeida e Calheiros de Souza Barbosa (ALMEIDA; BARBOSA, 2025), a arquitetura difere principalmente quanto à organização do código e ao acoplamento entre etapas: no PFC 2025, leitura de imagem, segmentação por cor, construção do grafo e execução dos algoritmos eram fortemente acopladas em *scripts* sequenciais. A Figura 2 resume as diferenças, destacando, na coluna da direita, os componentes efetivamente concluídos nesta segunda versão (segmentação, RRT e simulação em ciclo fechado).

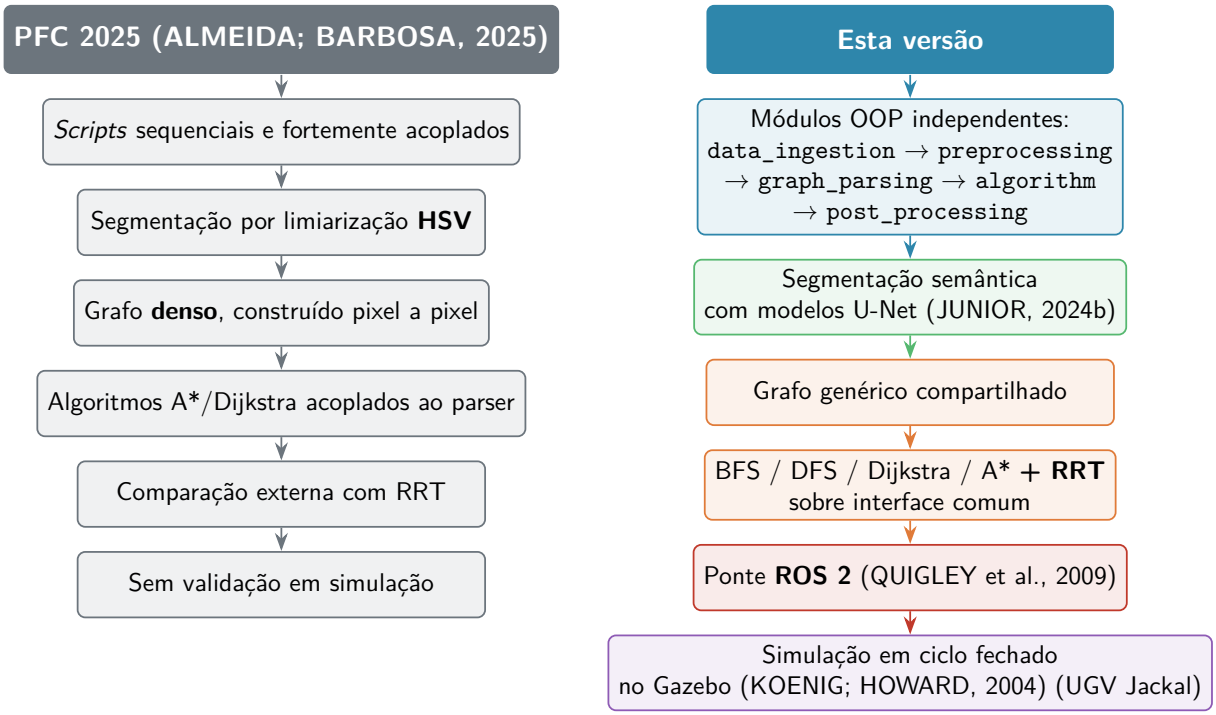


Figura 2 – Comparativo da estrutura entre o PFC 2025 (ALMEIDA; BARBOSA, 2025) e esta versão. Em destaque, os três aprimoramentos: segmentação semântica, inclusão do RRT e simulação em ciclo fechado.

3.2 Construção dos Grafos

A construção dos grafos toma como entrada o mapa de navegabilidade gerado pela segmentação (Seção 2.6). O *parser* identifica como nós os pixels marcados como transitáveis e estabelece arestas entre vizinhos imediatos, conforme a conectividade escolhida (4 ou 8 vizinhos).

Sobre a definição de grafo $G = (V, E)$ da Seção 2.3, adota-se neste projeto uma representação enriquecida, mais rica do que a utilizada no projeto anterior, composta por:

- ***Node embeddings***: vetor (x, y, w) para cada nó, com as coordenadas espaciais (x, y) do pixel e o peso de transitabilidade w;
- ***Edge index***: tensor de dimensão $(2, |E|)$ com os pares (origem, destino) de cada aresta, seguindo a convenção do PyTorch Geometric;
- ***Edge embeddings***: vetor (c_w, d_u) para cada aresta, com o custo ponderado c_w e a distância euclidiana d_u, definidos a seguir.

O peso de transitabilidade e os custos das arestas concretizam a função de pesos $w : E \to \mathbb{R}_{\geq 0}$ introduzida na Seção 2.4, penalizando trajetórias próximas a obstáculos.

Seja $d(v)$ a distância euclidiana de um nó v ao obstáculo mais próximo, obtida pela transformada de distância (`scipy.ndimage.distance_transform_edt`); sejam r (`robot_size`) a folga mínima exigida e L (`weight_limit`) o raio do campo de proximidade. O peso de um nó é dado por

$$w(v) = \begin{cases} \infty, & \text{se } d(v) \leq r \quad \text{(nó bloqueado)}, \\ L - d(v), & \text{se } r < d(v) < L, \\ 0, & \text{se } d(v) \geq L. \end{cases}$$

Ou seja, o peso cresce à medida que o nó se aproxima de um obstáculo, e os nós a menos de r pixels de um obstáculo são descartados. Para uma aresta $e = (u, v)$, seja $\delta_{uv} \in \{1,\ \sqrt{2}\}$ o comprimento do passo (1 para vizinhos ortogonais e $\sqrt{2}$ para diagonais). Os dois custos armazenados no *edge embedding* são

$$d_u = \delta_{uv}, \qquad c_w = \delta_{uv} + w(v),$$

isto é, d_u é a distância geométrica pura (utilizada na busca não ponderada) e c_w é o custo ponderado (utilizado na busca ponderada), que soma ao comprimento do passo o peso de proximidade do nó de destino. É esse custo c_w que penaliza a proximidade de obstáculos e produz caminhos mais afastados de regiões de risco.

3.3 Aprimoramento 1: Inclusão do Algoritmo RRT

O RRT (Seção 2.5) foi adicionado ao módulo `algorithm` sob a mesma interface `PathAlgorithm` dos demais planejadores, sendo registrado na fábrica ao lado de BFS, DFS, Dijkstra e A*. A implementação segue de perto a referência de Junior (JUNIOR, 2025), porém integrada à arquitetura do projeto:

- **Espaço contínuo sobre o mapa.** Ao contrário dos algoritmos de busca em grafos, que operam sobre a estrutura discreta, o RRT opera em coordenadas contínuas de pixel. O espaço livre, as coordenadas de origem/destino e a folga aos obstáculos são obtidos a partir do próprio `Graph`: um pixel é livre quando é um nó transitável e está suficientemente afastado de obstáculos (peso finito, segundo a transformada de distância). Assim, o RRT respeita a *mesma folga* (`robot_size`) dos demais algoritmos.

- **Verificação de colisão.** Segue a convenção de (JUNIOR, 2025): amostram-se cerca de 100 pontos ao longo do segmento entre duas posições, exigindo-se que todos recaiam sobre o espaço livre. A amostragem é feita de forma paramétrica, o que evita a divisão por zero da implementação de referência em segmentos verticais.

- **Compatibilidade com o pós-processamento.** Cada *waypoint* da árvore é associado ao nó de grafo mais próximo, de modo que os pós-processadores existentes (geração de *waypoints* e desenho sobre a imagem) funcionem sem alteração. O custo reportado é o *comprimento geométrico* do caminho (soma das distâncias euclidianas entre *waypoints*), e não o custo ponderado dos algoritmos de busca. Os dois não são diretamente comparáveis.
- **Parâmetros.** *Step size* (padrão 20 px), número máximo de iterações, taxa de amostragem do objetivo (*goal biasing*) e semente aleatória (para reprodutibilidade). São expostos tanto na linha de comando quanto na interface gráfica de teste.

3.4 Aprimoramento 2: Implementação da Segmentação Semântica

O módulo `preprocessing` passou a executar efetivamente os modelos de (JUNIOR, 2024b). Cada modelo é uma rede U-Net (RONNEBERGER; FISCHER; BROX, 2015) com um *backbone* convolucional (ResNet 50/101, EfficientNet-B4/B5 ou MobileNet V2), que produz, para cada pixel, uma distribuição sobre as 23 classes do *Semantic Drone Dataset*. O processamento compreende:

1. **Pré-processamento da imagem.** Conversão para RGB, normalização estatística com a média e o desvio-padrão da ImageNet (coerentes com o treinamento) e, como o codificador reduz a resolução por um fator de 32, preenchimento (*padding*) das bordas até o múltiplo de 32 mais próximo, recortado após a inferência.
2. **Inferência e agrupamento de classes.** A classe de cada pixel é obtida por *argmax*; as classes são então agrupadas em transitável e não transitável, considerando-se transitáveis as classes de solo (*paved-area*, *dirt*, *grass*, *gravel*), segundo critérios herdados de (JUNIOR, 2024a).
3. **Pós-processamento opcional da máscara.** Filtragem por confiança da predição, redimensionamento e limpeza morfológica (fechamento/abertura e remoção de componentes conexos pequenos). Estas etapas são configuráveis e, por padrão, desativadas, de modo que a saída de fábrica corresponda à máscara bruta do modelo.
4. **Geração do mapa de navegabilidade.** Aplicação da paleta de cores de (JUNIOR, 2025), produzindo o mapa consumido pelo `graph_parsing`.

A seleção do modelo em tempo de execução é um hiperparâmetro do *pipeline*, o que permite trocar o *backbone* sem alterar os demais módulos.

3.5 Aprimoramento 3: Integração com ROS 2 e Simulação no Gazebo

O terceiro aprimoramento fecha o ciclo entre planejamento e execução. Ele é composto por dois lados: a *publicação* a partir do *pipeline* de planejamento e o *consumo* pelo robô simulado.

3.5.1 Publicação: a ponte `ros_bridge`

O módulo `post_processing` foi estendido com uma ponte ROS 2 (`ros_bridge`) que publica dois tópicos: a imagem aérea anotada com a trajetória (`sensor_msgs/Image`) e o caminho como uma sequência de poses (`nav_msgs/Path`). Como os *waypoints* são expressos em coordenadas de pixel, a ponte os converte para metros por meio de um fator de escala (*Ground Sampling Distance*, metros por pixel), alinhando a trajetória ao referencial do mundo simulado; além disso, o eixo vertical da imagem é invertido para respeitar a convenção de referenciais do ROS. Utiliza-se uma política de QoS *transient local* (mensagem retida) para que o robô receba o caminho mesmo que se conecte após a publicação.

3.5.2 Consumo: o pacote `ugv_bridge` e o UGV simulado

Do lado do robô, um pacote ROS 2 (`ugv_bridge`) reúne os nós que fazem o UGV simulado percorrer a trajetória:

- `path_follower_node`: seguidor de *waypoints* para um robô de tração diferencial. A partir da odometria e da `nav_msgs/Path` recebida, publica comandos de velocidade (`geometry_msgs/Twist`), orientando-se para o próximo ponto e avançando quando alinhado, até alcançar o objetivo;
- `tf_odom_bridge_node`: republica, quando necessário, a transformação de pose (TF) como odometria (`nav_msgs/Odometry`), formato esperado pelo seguidor;
- `image_monitor_node`: registra a imagem anotada publicada pela ponte.

Como plataforma de demonstração, adotou-se o UGV *Clearpath Jackal* simulado no Gazebo, iniciado por um *launch file* de integração que sobe a simulação e os nós do `ugv_bridge`. Nessa configuração, a odometria filtrada é fornecida por um *Extended Kalman Filter* (EKF) que funde odometria das rodas e IMU. O fluxo completo é, portanto: o *pipeline* planeja e publica o caminho → o `path_follower_node` converte o caminho em comandos de velocidade → o Jackal percorre a trajetória no Gazebo até o destino.

3.6 Pós-processamento e Formato de Saída

Além da ponte ROS 2, o módulo `post_processing` mantém dois processadores: o `SimpleWaypointProcessor`, que converte a sequência de nós em *waypoints* com coordenadas (x, y), ângulo de proa θ e tipo de movimento (encapsulados na *dataclass* `RobotPath`); e o `ImageDrawingPostProcessor`, que desenha os caminhos sobre a imagem original, com cores distintas por algoritmo e marcadores de início e fim.

3.7 Coleta dos Dados

Para os experimentos, são utilizadas imagens RGB de alta resolução obtidas por drone, em cenários representativos de operação do UGV (caminhos pavimentados, vegetação, praças e obstáculos diversos), incluindo imagens da Quinta da Boa Vista. Os modelos de segmentação adotados já foram treinados em (JUNIOR, 2024b) sobre o *Semantic Drone Dataset* (Institute of Computer Graphics and Vision, TU Graz, 2019) e são aqui avaliados sobre essas imagens reais.

4 RESULTADOS E DISCUSSÃO

Neste capítulo, apresentamos e discutimos os resultados obtidos com os três aprimoramentos. Salvo indicação em contrário, os experimentos foram executados sobre imagens da Quinta da Boa Vista, com inferência em GPU (Apple Silicon, *backend* MPS).

4.1 Comparação dos Modelos de Segmentação

A Tabela 2 reúne, para os modelos com métricas publicadas em (JUNIOR, 2024b), a mIoU e a acurácia no conjunto de teste do *Semantic Drone Dataset*, além da fração de área classificada como transitável e do tempo de inferência medidos sobre uma imagem representativa redimensionada para 1024×768 pixels. A implementação suporta ainda o *backbone* MobileNet V2, aqui omitido da comparação quantitativa por não haver métricas de referência (mIoU/acurácia) publicadas.

Tabela 2 – Comparação dos modelos de segmentação. mIoU e acurácia conforme (JUNIOR, 2024b); área transitável e tempo de inferência medidos neste trabalho (1024×768, *backend* MPS).

Modelo	**mIoU**	**Acurácia**	**Área transit. (%)**	**Tempo (s)**
ResNet 50	0,3933	0,8244	65,5	0,72
ResNet 101	0,4032	0,8720	67,5	0,52
EfficientNet-B4	0,5012	0,9120	61,1	0,71
EfficientNet-B5	0,4819	0,9025	69,4	0,57

Os modelos EfficientNet apresentam as melhores métricas de referência, com destaque para o EfficientNet-B4 (maior mIoU, 0,5012). Os tempos de inferência são todos inferiores a um segundo na resolução avaliada, sem diferença expressiva entre os *backbones*, o que torna a mIoU o principal critério de escolha. A Figura 3 mostra, para o EfficientNet-B4, a máscara de área transitável sobreposta à imagem de entrada: o pátio pavimentado e a vegetação rasteira são corretamente identificados como transitáveis, ao passo que o coreto, as edificações, as árvores densas e os veículos são excluídos.

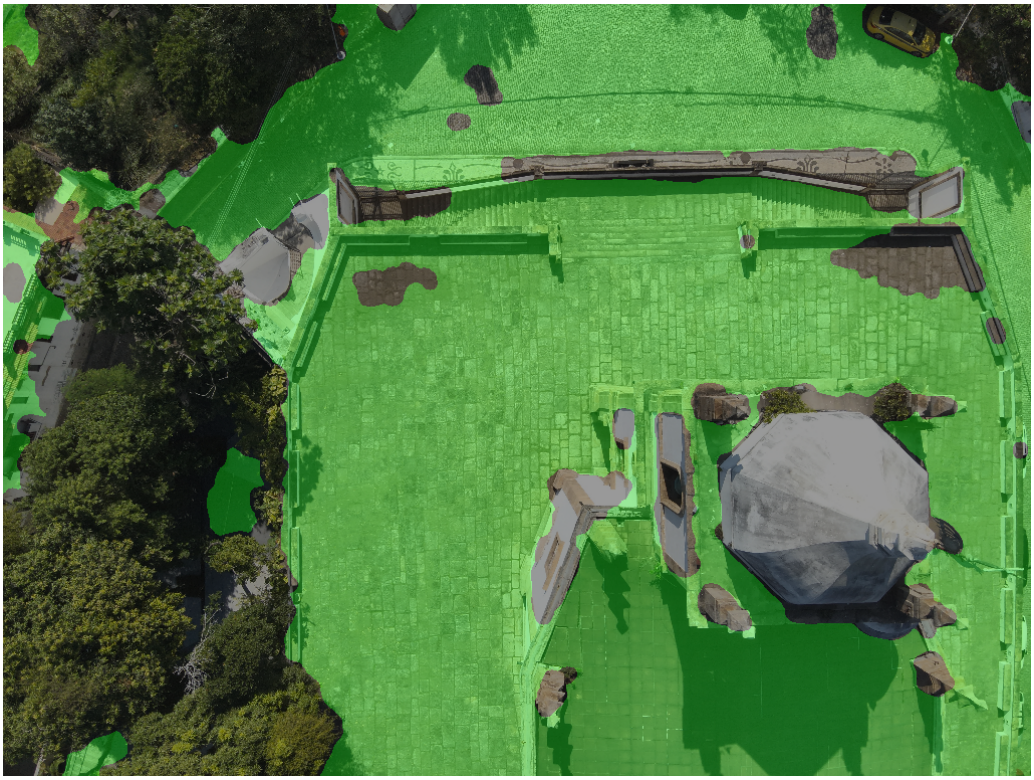


Figura 3 – Segmentação com o modelo EfficientNet-B4: em verde, a região classificada como transitável, sobreposta à imagem aérea de entrada.

4.2 Desempenho dos Algoritmos de Planejamento

A Tabela 3 compara os cinco algoritmos sobre o mesmo grafo, gerado a partir de uma imagem redimensionada para 1000×750 pixels (cerca de $4{,}4 \times 10^5$ nós e $3{,}3 \times 10^6$ arestas), com os mesmos pontos de origem e destino. Para os algoritmos de busca, o custo reportado é o custo *ponderado* c_w (que penaliza a proximidade de obstáculos, conforme a ponderação definida na Seção 3.2); para o RRT, é o comprimento geométrico do caminho.

Tabela 3 – Comparação dos algoritmos sobre o mesmo mapa (1000×750; mesmos início e fim). †Para o RRT, o custo é o comprimento geométrico, não comparável ao custo ponderado dos demais.

Algoritmo	**Nós no caminho**	**Custo**	**Nós expl. / iter.**	**Tempo (s)**	**Dist. real (px)**
BFS	351	12.838,8	179.546	0,131	398,0
DFS	65.282	1.314.372,7	248.479	0,159	92.241,7
Dijkstra	879	1.462,2	228.525	0,392	990,7
A*	879	1.462,2	203.472	0,426	990,7
RRT	30	931,6†	289	**0,010**	931,8

Os resultados são coerentes com a teoria e com o observado em (ALMEIDA; BARBOSA, 2025):

- **A* e Dijkstra** produzem exatamente o mesmo caminho ótimo ponderado (custo 1.462,2), mas o A* explora menos nós (203.472 contra 228.525) graças à heurística, confirmando a preferência pelo A* já apontada em (ALMEIDA; BARBOSA, 2025);
- A **BFS** encontra o caminho com menor número de arestas (menor distância real, 398,0 px), porém, por ignorar os pesos, apresenta o maior custo ponderado (12.838,8);
- A **DFS** confirma seu caráter não ótimo, produzindo um caminho extremamente longo (65.282 nós; distância real duas ordens de grandeza maior);
- O **RRT** é, de longe, o mais rápido (0,010 s, apenas 289 iterações) e produz um caminho curto em comprimento geométrico, porém irregular e não ótimo, e sujeito à variabilidade estocástica entre execuções.

4.3 Validação no Simulador Gazebo

A integração descrita na Seção 3.5 foi verificada em execução: a partir do *pipeline*, a trajetória planejada é publicada nos tópicos ROS 2 e o UGV *Clearpath Jackal* simulado no Gazebo a percorre efetivamente, seguindo os *waypoints* por meio do `path_follower_node` até alcançar o ponto de destino. Confirma-se, assim, o fechamento do ciclo entre planejamento e execução, objetivo central desta versão.

A avaliação quantitativa da simulação (aderência da trajetória executada ao caminho planejado, comportamento dinâmico do veículo e impacto da resolução do grafo e do fator de escala em metros por pixel sobre a execução) encontra-se em andamento e será consolidada na versão final do trabalho.

Do ponto de vista de dados, o simulador recebe como **entrada** o modelo do robô e do mundo (geometria, dinâmica e sensores) e os comandos de velocidade (`geometry_msgs/Twist`, no tópico `cmd_vel`) gerados pelo seguidor de caminho; como **saída**, publica o estado simulado do veículo (em especial a odometria `nav_msgs/Odometry` e as transformações de referencial, TF), além de eventuais fluxos de sensores. É essa odometria que realimenta o `path_follower_node`, fechando a malha de controle. A Figura 4 mostra o UGV Clearpath Jackal em um mundo simulado no Gazebo, com o painel de teleoperação publicando no tópico `cmd_vel` utilizado pela integração.



Figura 4 – UGV Clearpath Jackal simulado em um mundo do Gazebo. À direita, o painel de teleoperação publicando comandos de velocidade (`geometry_msgs/Twist`) no tópico `cmd_vel` consumido pela integração.

5 CONCLUSÃO

Esta segunda versão do projeto consolidou, sobre a arquitetura modular definida na primeira entrega, três aprimoramentos que transformaram o sistema de uma proposta com algoritmos-base em um *pipeline* completo, da imagem aérea à execução simulada.

O primeiro aprimoramento foi a inclusão do algoritmo **RRT** ao catálogo de planejadores, sob a mesma interface dos demais, permitindo comparar diretamente o planejamento por amostragem com a busca em grafos. Os experimentos reproduziram, de forma integrada, as conclusões de (ALMEIDA; BARBOSA, 2025): o A* combina otimalidade e eficiência (mesma solução do Dijkstra, com menos nós explorados), enquanto o RRT é muito mais rápido, porém não ótimo e sujeito à variabilidade estocástica.

O segundo aprimoramento foi a **implementação efetiva da segmentação semântica**, com os modelos U-Net pré-treinados de (JUNIOR, 2024b), permitindo gerar mapas de navegabilidade diretamente a partir de imagens RGB reais de drone, em substituição às abordagens baseadas em segmentação por cor. Entre os modelos avaliados, o EfficientNet-B4 apresentou a melhor mIoU, com tempo de inferência inferior a um segundo.

O terceiro aprimoramento foi a **integração com ROS 2 e a simulação do movimento no Gazebo**, na qual um UGV simulado (Clearpath Jackal) percorre efetivamente a trajetória planejada. Com isso, o sistema deixou de apenas produzir *waypoints* compatíveis e passou a fechar o ciclo entre planejamento e execução, oferecendo uma camada concreta de validação antes de qualquer implantação em campo.

5.1 Perspectivas Futuras

Como continuidade, destacam-se: (i) a consolidação da **avaliação quantitativa no Gazebo**, medindo a aderência da trajetória executada ao caminho planejado e o efeito da resolução do grafo e do fator metros-por-pixel; e (ii) a extensão do sistema para o tratamento de **ambientes dinâmicos**, em que obstáculos podem aparecer, desaparecer ou se mover ao longo da execução, exigindo replanejamento em tempo real do caminho.

6 CRONOGRAMA

A Figura 5, a seguir, apresenta o cronograma detalhado das atividades realizadas e planejadas ao longo do projeto.

Atividade	Março	Abril	Maio	Junho	Julho	Agosto	Setembro	Outubro
Revisão de Literatura								
Revisão do PFC 2025								
Revisão de Algoritmos para imagens								
Avaliação de algoritmos para nuvens de pontos								
Levantamento de Requisitos para nuvens de pontos								
Implementação de novos algoritmos								
Aplicação dos novos algoritmos								
Coleta e avaliação de resultados								
Simulação em ambiente virtual								
Simulação em ambiente real								
Documentação								

Figura 5 – Cronograma de atividades

Referências

ALMEIDA, J. V. C.; BARBOSA, T. C. S. *Planejamento de trajetória para robô terrestre, utilizando otimização em grafos*. Rio de Janeiro: [s.n.], 2025. Projeto de Final de Curso — Instituto Militar de Engenharia, Engenharia da Computação. 9, 14, 15, 16, 17, 18, 20, 21, 25, 26, 32, 33, 35

CHEN, W.-K. *Graph Theory and Its Engineering Applications*. [S.l.]: World Scientific, 1997. v. 5. 18, 19

CHEN, X.; ZHAO, M.; YIN, L. Dynamic path planning of the UAV avoiding static and moving obstacles. *Journal of Intelligent & Robotic Systems*, Springer, v. 99, p. 909–931, 2020. 18

CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge, MA: MIT Press, 2009. 19

DEO, N. *Graph Theory with Applications to Engineering and Computer Science*. [S.l.]: Courier Dover Publications, 2016. 19

DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, p. 269–271, 1959. 20

FINN, A.; SCHEDING, S. Developments and challenges for autonomous unmanned vehicles. *Intelligent Systems Reference Library*, Springer, v. 3, p. 128–154, 2010. 18

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, IEEE, v. 4, n. 2, p. 100–107, 1968. 20

Institute of Computer Graphics and Vision, TU Graz. *Semantic Drone Dataset*. 2019. Disponível em: <http://dronedataset.icg.tugraz.at/>. Acesso em: ago. 2026. 7, 8, 15, 21, 30

JUNIOR, F. L. *Cooperação entre Plataformas Robóticas Terrestres e Aéreas para aquisição, rastreamento e perseguição de alvos, utilizando Visão Computacional e Inteligência Artificial*. Tese (Doutorado) — Instituto Militar de Engenharia, Rio de Janeiro, 2024. 14, 16, 21, 28

JUNIOR, F. L. *Semantic Segmentation Models*. 2024. Repositório GitHub: <https://github.com/fabioluiz1987/semantic_segmentation_models>. Acesso em: ago. 2026. 7, 8, 10, 15, 16, 17, 21, 25, 26, 28, 30, 31, 35

JUNIOR, F. L. *RRT sobre mapa de navegação*. 2025. Repositório GitHub: <https://github.com/fabioluiz1987/RRT_sobre_mapa_de_navegacao>. Acesso em: ago. 2026. 17, 22, 27, 28

KOENIG, N.; HOWARD, A. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. [S.l.]: IEEE, 2004. v. 3, p. 2149–2154. 7, 8, 16, 23, 25, 26

LAVALLE, S. M. *Rapidly-exploring random trees: A new tool for path planning*. [S.l.], 1998. 15, 20

MADRIDANO, Á. et al. Trajectory planning for multi-robot systems: Methods and applications. *Expert Systems with Applications*, Elsevier, v. 173, p. 114660, 2021. 18

MOORE, E. F. The shortest path through a maze. *Proceedings of the International Symposium on the Theory of Switching*, Harvard University Press, p. 285–292, 1959. 19

QUIGLEY, M. et al. ROS: an open-source robot operating system. In: *ICRA Workshop on Open Source Software*. [S.l.: s.n.], 2009. 7, 8, 16, 22, 25, 26

RONNEBERGER, O.; FISCHER, P.; BROX, T. U-Net: Convolutional networks for biomedical image segmentation. In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. [S.l.]: Springer, 2015. p. 234–241. 21, 28

XU, T. Recent advances in rapidly-exploring random tree: A review. *Heliyon*, Elsevier, 2024. 20